

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3

Выполнил:

Каменев Матвей

Группа J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Ход работы

1. Инициализация и структура проекта

Приложение переписано на Vue 3 с использованием сборщика Vite (@vitejs/plugin-vue). Зависимости: vue, vue-router, pinia, axios, bootstrap.

Исходный код организован по назначению:

src/api: axios-инстанс и классы API по ресурсам;

src/router: маршрутизация;

src/stores: Pinia-сторы;

src/composables: переиспользуемая логика;

src/layouts, src/components, src/views: макет, компоненты и страницы.

Все данные, которые в ЛР1–ЛР2 были захардкожены в HTML, перенесены в db.json и загружаются с сервера: пользователи (users), заказы (orders), дата поинты (datapoints), принятые заказы (annotations), результаты разметки (results), профили (profiles) и проекты заказчиков (projects). Связи между коллекциями задаются полями userId и orderId.

2. Маршрутизация (src/router/index.js)

Подключён vue-router. Отдельные HTML-страницы заменены маршрутами:

/ главная страница с каталогом заказов;

/profile личный кабинет;

/projects/:id страница проекта;

/orders/:id/annotate страница разметки заказа;

`/:pathMatch(.*)*` перенаправление неизвестных адресов на главную.

Компоненты страниц загружаются лениво через динамический `import()`. У маршрутов задано поле `meta.requiresAuth`, а глобальный навигационный гвард `beforeEach` перенаправляет неавторизованного пользователя на главную, передавая исходный адрес в параметре запроса:

```
router.beforeEach((to) => {
  const auth = useAuthStore()

  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { name: 'home', query: { redirect: to.fullPath } }
  }
})
```

При наличии параметра `redirect` компонент `LoginModal` автоматически открывает окно входа, а после успешного входа пользователь возвращается на запрошенную страницу.

3. Работа с API через axios (src/api)

В `instance.js` создан `axios`-инстанс с базовым адресом `http://localhost:3000`. Интерсептор запросов добавляет заголовок `Authorization: Bearer <токен>`, а интерсептор ответов (в `main.js`) при ошибке 401 выполняет выход и переход на главную.

Запросы сгруппированы в классы по ресурсам: `AuthApi`, `OrdersApi`, `AnnotationsApi`, `ProfilesApi`, `ProjectsApi`, `DatapointsApi`, `ResultsApi`. Каждый класс получает инстанс в конструкторе, а готовые экземпляры экспортируются из `api/index.js`.

Внешнее API курсов валют `open.er-api.com` из ЛР2 вынесено в отдельный `axios`-инстанс и класс `RatesApi`.

4. Управление состоянием (src/stores)

Состояние приложения хранится в `Pinia`-сторах:

auth токен и пользователь (с сохранением в localStorage),
вычисляемое свойство isAuthenticated, действия login,
register, logout;
orders, annotations, profile, projects загрузка каталога заказов,
принятых заказов, данных профиля и проектов;
task текущая задача разметки: заказ, его дата поинты и уже
сохранённые результаты. Вычисляемое свойство
currentDatapoint возвращает первый неразмеченный дата поинт,
progress номер текущего и общее количество. Данные задачи
загружаются параллельно через Promise.all.

5. Компонентный подход

Интерфейс разделён на компоненты по областям:

layouts/BaseLayout.vue общий макет (навбар, слот для
содержимого, футер);

common: NavbarComponent, FooterComponent,
CurrencySwitcher, PriceLabel (цена с конвертацией),
SvgSprite (SVG-спрайт из ДЗ4);

auth: LoginModal, RegisterModal;

home, orders: HeroSection, OrderCard, OrdersFilter;

profile, project: ProfileHeader, StatCard, ProjectHeader,
DatapointCard, AnnotatorsTable;

annotation: AnnotationHeader, TaskProgress, LabelChoice
(выбор класса через defineModel), ConfirmButton, BoundingBox
(события delete и resize-start).

Инструменты разметки из ЛР1 переписаны как четыре компонента с
единым интерфейсом: ImageClassificationTask,
TextClassificationTask, DetectionTask и NerTask принимают
props datapoint, labels, saving и генерируют событие submit с

результатом. Страница `AnnotationView` выбирает нужный компонент по типу заказа через `<component :is>`;

6. Composables (src/composables)

Повторяющаяся логика вынесена в отдельные функции:

`useCurrency`: курсы валют, выбранная валюта и форматирование цен.

`useRequest`: обёртка над асинхронным запросом с реактивными `loading` и `error` и опцией немедленного запуска; используется на всех страницах;

`useModal`: показ и скрытие модальных окон Bootstrap по идентификатору;

`useAuthForm`: общая логика форм входа и регистрации: вывод ошибки, закрытие окна и переход по `redirect`;

7. Новая функциональность

Помимо переноса существующих страниц, разметка стала полноценной: кнопка «Подтвердить» отправляет результат на сервер (`POST /600/results`) и переходит к следующему пункту, а после разметки всех элементов выводится соответствующее сообщение. Размечать можно только принятый заказ, для непринятого на странице разметки показывается кнопка «Принять». Доработки из домашних работ (ARIA-атрибуты, тёмная тема, SVG-спрайт) сохранены.

Вывод

В ходе лабораторной работы приложение платформы для аннотации данных было мигрировано с набора статических HTML-страниц на одностраничное приложение на Vue 3. Подключён `vue-router` с ленивой загрузкой страниц и навигационным гвардом, работа с API переведена на `axios` с интерсепторами для авторизации, состояние вынесено в `Pinia`-сторы, а все данные перенесены на моковый сервер.